#11 캠 등록 — 코드 상세 설명서

출처: Notion `[#11] 코드리뷰 · 캠 등록` (PR #109). 아래 코드는 Notion에 적힌 코드다.

1. 전체 흐름

```
관리자: 구역·이름·입력 방식·영상 주소
   → CameraController           경비원: 요청 받기, 201 응답
   → CameraRegisterService      센터장: 순서대로 시키기, 전부 되거나 전부 취소
        ├→ ZoneService.requireActiveZone      구역이 있고 안 지워졌나? 아니면 404
        ├→ CameraRepository.existsBy...       그 구역에 살아 있는 캠 있나? 있으면 409
        ├→ CameraSourceUriValidator           영상 주소 규칙 맞나? 아니면 400
        └→ CameraRepository.save              camera 저장 (STOPPED)
   → 201 캠 정보
```

2. 명세가 요구한 것 vs 구현

명세 (FNC-A-018)	구현
1. 구역 존재·삭제 여부 확인	`ZoneService.requireActiveZone()` → 404 `ZONE_NOT_FOUND`
2. 그 구역에 삭제되지 않은 캠이 있으면 거부	`existsByZoneIdAndDeletedAtIsNull` → 409 `ZONE_HAS_CAMERA`
3. 영상 주소 규칙 검사	`CameraSourceUriValidator` → 400 `INVALID_INPUT`
4. 저장: status=STOPPED, run_requested=0	`Camera` 생성자가 정함
5. 작업 기록(CAMERA_CHANGE, op: CREATE)	**미구현** (#6 대기)
API: 201 캠 정보(status=STOPPED)	`CameraResponse`
입력값 규칙: 캠 이름 1~50자, 영상 주소	`@NotBlank`·`@Size`, `CameraSourceUriValidator`

`ZONE_NOT_FOUND`는 명세에 코드가 없어서 규약 예시를 따랐다.

3. 코드

3-1. DB 표: V5__create_camera.sql

```
CREATE TABLE camera (
    id BIGINT NOT NULL AUTO_INCREMENT COMMENT '캠 번호',
    zone_id BIGINT NOT NULL COMMENT '담당 구역(zone.id). 구역당 삭제되지 않은 캠 1대(앱에서 확인)',
    name VARCHAR(50) NOT NULL COMMENT '캠 이름',
    source_type VARCHAR(10) NOT NULL COMMENT '입력 방식: LIVE / DUMMY',
    source_uri VARCHAR(255) NOT NULL COMMENT '영상 주소: LIVE는 RTSP 주소, DUMMY는 공유 폴더 기준 상대경로',
    run_requested TINYINT(1) NOT NULL DEFAULT 0 COMMENT '1이면 가동을 원하는 상태(가동하면 1, 정지하면 0). 실제 상태(s
tatus)와 따로 둠',
    status VARCHAR(20) NOT NULL DEFAULT 'STOPPED' COMMENT '실제 상태: STOPPED / PREPARING / RUNNING / ERRO
R',
    progress TINYINT NOT NULL DEFAULT 0 COMMENT '분석 준비 진행률(0~100)',
    error_message VARCHAR(500) NULL COMMENT '마지막 오류 내용',
    first_started_at DATETIME NULL COMMENT '처음 가동된 시각(NULL이면 가동 이력 없음)',
    deleted_at DATETIME NULL COMMENT '삭제 시각(NULL이면 사용 중)',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '등록 시각',
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '마지막 수정
```

```
시각',
    CONSTRAINT camera_pk PRIMARY KEY (id),
    CONSTRAINT camera_fk_zone FOREIGN KEY (zone_id) REFERENCES zone (id)
) ENGINE = InnoDB;

CREATE INDEX camera_ix_zone_deleted ON camera (zone_id, deleted_at);
CREATE INDEX camera_ix_run_requested ON camera (run_requested);
```

부분	쉬운 뜻
`FOREIGN KEY (zone_id) REFERENCES zone (id)`	캠은 반드시 실제 있는 구역에 붙어야 한다는 약속
`run_requested TINYINT(1) DEFAULT 0`	"켜기를 원하는 상태"인지 적는 스위치(0/1). 실제 상태(`status`)와 따로 둔다
`status DEFAULT 'STOPPED'`	분석 서버가 보고한 실제 상태. 처음엔 정지
`progress TINYINT`	더미 영상 분석 준비 진행률(0~100)
`first_started_at`	처음 켜진 시각. 비어 있으면 한 번도 안 켜진 카메라 (나중에 "완전 삭제 가능"을 판단할 때 쓴다)
`camera_ix_zone_deleted (zone_id, deleted_at)`	"이 구역의 살아 있는 캠"을 빨리 찾는 색인
구역당 1대에 UNIQUE가 없다	명세가 "앱에서 확인"이라고 했다. 삭제된 카메라는 여러 대 남아야 하기 때문이다

3-2. 표의 Java 버전: Camera.java (common)

```
@Entity
@Table(name = "camera")
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class Camera {

    private static final int INITIAL_PROGRESS = 0;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // zone 도메인의 엔티티를 직접 들지 않고 번호만 둔다
    @Column(name = "zone_id", nullable = false)
    private Long zoneId;

    @Column(nullable = false, length = 50)
    private String name;

    @Enumerated(EnumType.STRING)
    @Column(name = "source_type", nullable = false, length = 10)
    private CameraSourceType sourceType;

    @Column(name = "source_uri", nullable = false, length = 255)
    private String sourceUri;

    @Column(name = "run_requested", nullable = false)
    private boolean runRequested;

    @Enumerated(EnumType.STRING)
    @Column(name = "status", nullable = false, length = 20)
    private CameraStatus status;

    @JdbcTypeCode(SqlTypes.TINYINT)
    @Column(name = "progress", nullable = false)
```

```
    private int progress;

    @Column(name = "error_message", length = 500)
    private String errorMessage;

    @Column(name = "first_started_at")
    private LocalDateTime firstStartedAt;

    @Column(name = "deleted_at")
    private LocalDateTime deletedAt;

    @Generated(event = EventType.INSERT)
    @Column(name = "created_at", insertable = false, updatable = false)
    private LocalDateTime createdAt;

    @Generated(event = {EventType.INSERT, EventType.UPDATE})
    @Column(name = "updated_at", insertable = false, updatable = false)
    private LocalDateTime updatedAt;

    /** 새 캠은 정지 상태(STOPPED), 가동 요청 없음(run_requested=0)으로 만든다(FNC-A-018 처리 순서 4). */
    public Camera(Long zoneId, String name, CameraSourceType sourceType, String sourceUri) {
        this.zoneId = zoneId;
        this.name = name;
        this.sourceType = sourceType;
        this.sourceUri = sourceUri;
        this.runRequested = false;
        this.status = CameraStatus.STOPPED;
        this.progress = INITIAL_PROGRESS;
    }

    /** 분석 서버가 보고한 상태로 바꾼다(FNC-A-025). progress가 null이면 기존 진행률을 유지한다. */
    public void updateStatus(CameraStatus status, Integer progress, String errorMessage) { ... }
}
```

부분	쉬운 뜻
`private Long zoneId`	구역 정보를 통째로 들지 않고 **구역 번호만** 가진다 (다른 도메인 엔티티 직접 사용 금지). #10의 `Zone`이 `dongId`만 가진 것과 같은 방식
`@JdbcTypeCode(SqlTypes.TINYINT)` (progress)	DB 칸이 작은 숫자(TINYINT)라서 자바에게도 알려 준다. 안 그러면 스프링이 시작할 때 "칸 종류가 다르다"고 검사에서 떨어뜨릴 수 있다
`Camera(zoneId, name, sourceType, sourceUri)` 생성자	새 카메라는 정지 상태·가동 요청 없음·진행률 0으로 만든다. 상태를 밖에서 정하지 못하게 해서 "정지 상태로 등록" 규칙을 생성자가 지킨다
`updateStatus(...)`	다른 팀원(#15 캠 상태 보고)이 이미 쓰고 있는 메서드. 내용을 그대로 두어 깨지지 않게 했다
필드 이름 (`zoneId`·`sourceType`·`runRequested`·`status`·`progress`·`errorMessage`·`deletedAt`)	다른 팀원의 테스트가 이 이름으로 값을 직접 넣는다 (`ReflectionTestUtils`). 이름을 바꾸지 않았다

3-3. enum 2개와 CameraRepository (common)

```
public enum CameraSourceType { LIVE, DUMMY }
public enum CameraStatus { STOPPED, PREPARING, RUNNING, ERROR }

public interface CameraRepository extends JpaRepository<Camera, Long> {

    Optional<Camera> findByIdAndDeletedAtIsNull(Long id);

    /** 그 구역에 삭제되지 않은 캠이 있는지 본다(구역당 1대, FNC-A-018 처리 순서 2). */
    boolean existsByZoneIdAndDeletedAtIsNull(Long zoneId);
}
```

`findByIdAndDeletedAtIsNull`은 다른 팀원의 임시 코드에 있던 메서드를 그대로 유지했다.
`existsByZoneIdAndDeletedAtIsNull`이 새로 생긴 질문이다: "이 구역 번호에 삭제 시각이 비어 있는(살아 있는) 캠이 있나?"

3-4. 요청·응답·오류 상자

```
// CameraCreateRequest — sourceUri의 입력 방식별 규칙은 service가 확인한다. 여기서는 비었는지와 길이만 본다.
public record CameraCreateRequest(
        @NotNull(message = "필수입니다.") Long zoneId,
        @NotBlank(message = "필수입니다.") @Size(max = 50, message = "50자 이하로 입력해 주세요.") String name,
        @NotNull(message = "필수입니다.") CameraSourceType sourceType,
        @NotBlank(message = "필수입니다.") @Size(max = 255, message = "255자 이하로 입력해 주세요.") String sourceU
ri) {}

// CameraResponse — id, zoneId, name, sourceType, sourceUri, status, progress, errorMessage, runRequested
// (Camera에서 옮겨 담는 from(camera) 정적 메서드 포함)

// ZoneHasCameraException
public class ZoneHasCameraException extends BusinessException {
    public ZoneHasCameraException(Long zoneId) {
        super(ErrorCode.ZONE_HAS_CAMERA, "이 구역에는 이미 캠이 있습니다. zoneId=" + zoneId);
    }
}

// InvalidSourceUriException — 영상 주소가 규칙을 어겼다(400 INVALID_INPUT)
public class InvalidSourceUriException extends BusinessException {
    public InvalidSourceUriException(String message) {
        super(ErrorCode.INVALID_INPUT, message);
    }
}

// ErrorCode 에 추가
ZONE_NOT_FOUND(HttpStatus.NOT_FOUND, "구역을 찾을 수 없습니다."),
ZONE_HAS_CAMERA(HttpStatus.CONFLICT, "이 구역에는 이미 캠이 있습니다."),
```

- `sourceType`이 없거나 LIVE/DUMMY가 아닌 값(예: WEBCAM)이면 요청을 읽는 단계에서 400 `INVALID_INPUT`이 난다.
- 영상 주소 규칙 위반은 `INVALID_INPUT`(400)으로 던진다. 명세가 "400"만 정했고 전용 코드는 없어서 공통 코드를 썼다. (리뷰 R6)

3-5. 영상 주소 심사관: CameraSourceUriValidator

```
@Component
public class CameraSourceUriValidator {

    private static final int MAX_LENGTH = 255;
    private static final String LIVE_PREFIX = "rtsp://";
    private static final String DUMMY_PREFIX = "dummy/videos/";
    private static final String DUMMY_SUFFIX = ".mp4";
    private static final String PARENT_PATH = "..";
```

```
    public void validate(CameraSourceType sourceType, String sourceUri) {
        if (sourceUri.length() > MAX_LENGTH) {
            throw new InvalidSourceUriException("영상 주소는 " + MAX_LENGTH + "자 이하로 입력해 주세요.");
        }
        switch (sourceType) {
            case LIVE -> {
                if (!sourceUri.startsWith(LIVE_PREFIX)) {
                    throw new InvalidSourceUriException("실시간(LIVE) 영상 주소는 " + LIVE_PREFIX + " 로 시작해야
합니다.");
                }
            }
            case DUMMY -> {
                if (!sourceUri.startsWith(DUMMY_PREFIX)
                        || !sourceUri.endsWith(DUMMY_SUFFIX)
                        || sourceUri.contains(PARENT_PATH)) {
                    throw new InvalidSourceUriException("더미(DUMMY) 영상 주소는 " + DUMMY_PREFIX + " 로 시작하고
"
                            + DUMMY_SUFFIX + " 로 끝나는 상대경로여야 하며 '..'를 쓸 수 없습니다.");
                }
            }
        }
    }
}
```

부분	쉬운 뜻
`sourceUri.length() > 255`	주소가 DB 칸(255자)보다 길면 거부. 입력 방식과 상관없이 먼저 본다
`case LIVE` : `startsWith("rtsp://")`	실시간 카메라는 주소가 반드시 `rtsp://` 로 시작(소문자). 대문자 `RTSP://` 나 `http://` 는 거부
`case DUMMY` : 세 조건	연습용 영상은 공유 폴더 안의 파일만 가리켜야 한다. ① `dummy/videos/` 로 시작 (절대경로 `/srv/...` 도 자동으로 거부) ② `.mp4` 로 끝 ③ `..` 없음 (폴더를 빠져나가 다른 파일을 가리키는 것 차단)
파일이 실제로 있는지는 안 본다	명세: "파일·스트림 실제 존재 여부는 가동할 때 분석 서버가 확인"
별도 클래스로 분리	캠 정보 수정(FNC-A-019)도 같은 규칙을 써야 해서 한 곳에 둔다
`switch (sourceType) { case LIVE -> ... }`	입력 방식에 따라 갈래를 나누는 문법

3-6. 센터장: CameraRegisterService (핵심)

```
@Service
@RequiredArgsConstructor
public class CameraRegisterService {

    private final ZoneService zoneService;
    private final CameraRepository cameraRepository;
    private final CameraSourceUriValidator sourceUriValidator;

    @Transactional
    public CameraResponse register(CameraCreateRequest request) {
        // 처리 순서 1: 구역 존재·삭제 여부 확인
        zoneService.requireActiveZone(request.zoneId());
        // 처리 순서 2: 그 구역에 삭제되지 않은 캠이 있으면 거부
        if (cameraRepository.existsByZoneIdAndDeletedAtIsNull(request.zoneId())) {
            throw new ZoneHasCameraException(request.zoneId());
        }
        // 처리 순서 3: 영상 주소 규칙 검사
        String sourceUri = request.sourceUri().trim();
        sourceUriValidator.validate(request.sourceType(), sourceUri);
        // 처리 순서 4: 저장(status=STOPPED, run_requested=0은 Camera 생성자가 정한다)
        Camera camera = cameraRepository.save(
```

```
                new Camera(request.zoneId(), request.name().trim(), request.sourceType(), sourceUri));
        // 처리 순서 5(작업 기록 CAMERA_CHANGE)는 작업 기록 저장(FNC-C-026, #6)이 들어오면 여기서 호출한다
        return CameraResponse.from(camera);
    }
}
```

줄	쉬운 뜻
`@Transactional`	메서드 전체가 한 묶음. 중간에 오류가 나면 저장한 것도 전부 취소된다
`zoneService.requireActiveZone(...)`	구역 도메인의 공개 서비스에게 "이 구역 있고 안 지워졌니?"라고 묻는다. 다른 영역의 저장소를 직접 쓰지 말라는 팀 규칙 때문에 서비스를 거친다. 아니면 404
`existsByZoneIdAndDeletedAtIsNull`	"이 구역에 살아 있는 캠이 있나?" 있으면 409. 삭제된 캠은 세지 않는다
`request.sourceUri().trim()`	주소 앞뒤 공백을 지운 다음 규칙을 검사하고 그 값을 저장한다 (내가 정한 것, 명세에 없음)
`new Camera(...)`	상태를 넘기지 않는다. 정지 상태·가동 요청 없음으로 만들어진다
`CameraResponse.from(camera)`	저장 후 DB가 정한 번호와 저장된 값을 응답 상자로 옮겨 담는다

3-7. 구역 확인 창구: ZoneService (zone 도메인)

```
@Service
@RequiredArgsConstructor
public class ZoneService {

    private final ZoneRepository zoneRepository;

    /** 구역이 있고 삭제되지 않았는지 확인한다. 아니면 404 ZONE_NOT_FOUND. */
    @Transactional(readOnly = true)
    public void requireActiveZone(Long zoneId) {
        if (!zoneRepository.existsByIdAndDeletedAtIsNull(zoneId)) {
            throw new ZoneNotFoundException(zoneId);
        }
    }
}
```

캠 도메인이 구역을 확인할 때 부르는 **구역 쪽 공개 창구**다. `ZoneRepository`에 `existsByIdAndDeletedAtIsNull`이 새로 생겼다. (#10 파일 수정) 팀원의 #18 브랜치에는 `ZoneQueryService`(면적 조회)가 같은 패키지에 있다. 이름이 달라 파일 충돌은 없지만 나중에 합칠지 정해야 한다. (리뷰 R9)

3-8. 경비원: CameraController

```
@RestController
@RequestMapping("/api/v1/manage/cameras")
@RequiredArgsConstructor
public class CameraController {

    private final CameraRegisterService cameraRegisterService;

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public CameraResponse register(@Valid @RequestBody CameraCreateRequest request) {
        return cameraRegisterService.register(request);
    }
}
```

`POST /api/v1/manage/cameras` (관리자 앱은 앞에 `/admin`이 붙는다). 성공하면 **201**. `@Valid`는 요청 상자의 검사(`@NotNull`·`@NotBlank`·`@Size`)를 한다. 컨트롤러는 받아서 넘기기만 한다.

4. 테스트 35개 (전부 통과)

테스트	확인하는 것
`register_liveCamera_createsStoppedCamera`	201과 캠 정보(STOPPED·progress 0·runRequested false), DB 행 값
`register_dummyCamera_created`	DUMMY 캠도 등록
`register_trimsNameAndUri`	이름·주소 앞뒤 공백 제거
`register_zoneNotFound`, `register_deletedZone`	없는 구역·삭제된 구역 → 404 `ZONE_NOT_FOUND`, 0건 저장
`register_zoneAlreadyHasCamera`	같은 구역 두 번째 캠 → 409 `ZONE_HAS_CAMERA`, 1건 유지
`register_otherZone_allowed`	다른 구역에는 각각 등록
`register_afterCameraDeleted_allowed`	캠이 삭제된 구역에 다시 등록
`register_liveUriNotRtsp_badRequest`	LIVE인데 `http://` → 400 `INVALID_INPUT`, 0건 저장
`register_dummyUriBreaksRule_badRequest`	DUMMY인데 `..` 포함 → 400, 0건 저장
`register_zoneCheckedBeforeUriRule`	없는 구역 + 잘못된 주소 → 404 (구역 확인이 먼저)
`register_blankName_badRequest` 외 2개	이름 비었음·50자 초과·주소 255자 초과 → 400 `INVALID_INPUT`
`register_invalidSourceType_badRequest`, `register_missingZoneId_badRequest`	sourceType 없음·WEBCAM, zoneId 없음 → 400
`CameraSourceUriValidatorTest` (19개)	LIVE 통과 3·거부 4, DUMMY 통과 3·거부 8, 255자까지 통과·256자 거부

테스트는 H2(MySQL 모드)에 #7의 샘플 경계 파일과 #10의 구역 등록을 써서 구역 2개를 만든 뒤 MockMvc로 요청한다. **테스트가 끝나면 `camera` → `zone` 순서로 행을 지운다.** 안 지우면 뒤에 도는 지역 테스트가 `region`을 지우다 외래키 오류로 19개 실패했다.

5. 왜 이렇게 만든 게 맞는가

결정	이유	근거
구역 확인 → 중복 → 주소 규칙 → 저장	명세의 순서와 오류 우선순위를 그대로 따름 (테스트로 고정)	FNC-A-018
엔티티·enum·저장소를 팀원의 임시 코드와 같은 이름으로	합칠 때 클래스만 갈아 끼우면 되고 그쪽 코드를 고칠 필요가 없도록	임시 코드의 "TODO(#11)" 주석
`Camera`는 `zoneId`(번호)만 보관, 구역 확인은 `ZoneService`를 거침	다른 도메인의 엔티티·repository 직접 사용 금지	package-structure.md
새 캠은 생성자가 정지 상태로 만든다	호출하는 쪽이 실수로 바꿀 수 없도록	FNC-A-018 처리 순서 4
주소 규칙 검사는 별도 컴포넌트	캠 수정(FNC-A-019)이 같은 규칙을 재사용	conventions.md
오류는 `BusinessException` + `ErrorCode`	공통 오류 처리가 응답을 만들어 줌	FNC-C-027

결정	이유	근거
새 마이그레이션 `V5`	dev에 합쳐진 V파일은 수정하지 않는다	db-migration 규칙

6. 선배가 물을 만한 질문과 답

질문	답
"팀원이 이미 `Camera` 임시 코드를 만들어 뒀는데 왜 또 만들었어?"	임시 코드에 "TODO(#11): 정식으로 만든다"라고 표시돼 있었다. 같은 클래스·필드·메서드 이름을 그대로 두고 칼럼만 채워서, 합칠 때 갈아 끼우면 되게 했다. 임시 브랜치에 #15~#18을 합쳐 컴파일이 통과하는 것도 확인했다.
"합치면 문제는 없어?"	컴파일은 통과한다. 다만 팀원 쪽 테스트 설정의 `create-drop`을 지우고 validate로 되돌리면 그쪽 통합 테스트 5개가 실패한다. 임시 칼럼만 채워 넣은 데이터가 정식 NOT NULL 칼럼과 안 맞아서다. 합친 뒤 테스트 데이터를 채워야 하고 담당자와 협의가 필요하다.
"구역 확인을 왜 `ZoneRepository`로 안 하고 `ZoneService`로 해?"	다른 도메인의 repository를 직접 쓰지 말라는 팀 규칙이다. 구역 도메인이 공개한 서비스를 거친다.
"왜 새 캠은 항상 정지 상태야?"	명세가 정했다. 실제 화면이 나오는지는 나중에 가동할 때 분석 서버가 확인한다. 생성자가 상태를 정해서 호출하는 쪽이 못 바꾼다.
"`run_requested`와 `status`는 왜 둘 다 있어?"	원하는 상태(켜 달라)와 실제 상태(분석 서버가 보고한 것)가 다를 수 있어서다.
"DUMMY 주소에서 `..`을 막는 이유는?"	공유 폴더 밖의 파일을 가리키는 것을 차단하려고. `dummy/videos/../../etc/passwd.mp4` 같은 주소를 막는다.
"파일이 실제로 있는지는 왜 안 봐?"	명세가 가동할 때 분석 서버가 확인하라고 정했다.
"구역당 1대를 왜 DB UNIQUE로 안 막아?"	삭제된 캠은 같은 구역에 여러 대 남아야 해서 명세가 앱 확인으로 정했다. 대신 동시 등록 시 둘 다 저장될 수 있다는 약점이 있다.
"마이그레이션이 왜 V5야?"	리뷰 요청으로 V4에서 바꿨다. #108이 V3, #107의 zone이 V4를 쓰기 때문이다. 나중에 #6이 같은 번호를 쓰면 병합하는 쪽이 올려야 한다.

7. 명세 대조 (FNC-A-018)

명세 항목	구현	확인
1. 구역 존재·삭제 여부 확인	`ZoneService.requireActiveZone`	`register_zoneNotFound`, `register_deletedZone`
2. 구역당 살아 있는 캠 1대	`existsByZoneIdAndDeletedAtIsNull`	`register_zoneAlreadyHasCamera`, `register_otherZone_allowed`, `register_afterCameraDeleted_allowed`
3. 영상 주소 규칙 검사	`CameraSourceUriValidator`	주소 규칙 400 테스트 2개 + 단위 테스트 19개
4. 저장(STOPPED, run_requested=0)	`Camera` 생성자	`register_liveCamera_createsStoppedCamera`
5. 작업 기록 (CAMERA_CHANGE)	**미구현** (#6 대기)	-

명세 항목	구현	확인
409 ZONE_HAS_CAMERA / 400 주소 규칙	`ZoneHasCameraException` / `InvalidSourceUriException`	위 테스트
201 캠 정보 (status=STOPPED)	`CameraResponse`	`register_liveCamera_createsStoppedCamera`